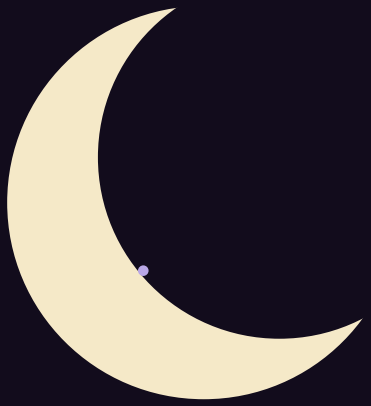
KIROWEEN 2026 | IDEA BOOK



Five Ideas From the Crypt

Pick one. Build it from Oct 30 to Dec 4.

Abbas Rahman & team | prepared 30 October 2026

1 2 3 4 5

BEFORE YOU PICK

What the rules ask for

The full rules went live on 30 October. These are the parts that shape which idea fits.

Deadline	Submissions close Friday 4 December 2026, 2:00 PM Pacific (Saturday 3:00 AM in Pakistan). No extensions.
Team	Up to 3 people. One person is the Representative who submits.
Free credits	Each member can request a one-time 4,000 Kiro credits through the Credit Request Form, by 1 December. Request on day one, allow 48 business hours.
Must use Kiro	Built with Kiro (IDE, CLI, Web or Crew). Weak Kiro use is grounds for disqualification. The repo must keep the **/.kiro** folder (specs, hooks, steering) at its root, never in .gitignore.
Deliverables	Working app URL judges can use for free, public GitHub repo with an open source license shown in the About section, video under 3 minutes on YouTube, text description, and a write-up of how Kiro was used.
New work only	Everything built after 30 October. Libraries and templates are fine but must be disclosed.
Bonus prizes	Best Student Project (every member must be a student), Best Kiro Crew App, Social Blitz (post starting with #KiroweenHackathon26), Blog post, Startup and Ghost of Kiroween Past.

How it is judged

Stage one is pass/fail: does it fit the theme and really use Kiro. Stage two scores five criteria, each worth 20%:

Criterion	What judges look for
Potential Impact	A real problem for a real audience, and something people would actually use.
Technological Implementation	Deep, varied use of Kiro: specs, hooks, steering, MCP, custom agents, property-based testing, powers, and experimentation.
Design	A complete, polished product, not a proof of concept.
Innovation	Novel concept, unusual technologies or public datasets combined in new ways.
Presentation	A clear, memorable video showing it working end to end.

Every idea in this book is designed around free tools and public data, and around a write-up that shows off the full range of Kiro features.

IDEA 1 | FRANKENSTEIN + COSTUME CONTEST

SÉANCE

Summon the ghosts of any codebase and ask them why the code is the way it is.

The idea

Every team loses its history. The person who wrote the payment retry logic left two years ago, the reason lives in a closed PR nobody can find, and new engineers spend weeks guessing. Séance turns any GitHub repository into a haunted mansion you can walk through. Each module is a room. Rooms are haunted by real problems: dead code, stale TODOs, flaky tests, files nobody has touched in years. Click a ghost and talk to it: it answers in the voice of the commit that created it, and every sentence cites the commit, PR or issue it came from.

What it does

- **The mansion:** a Three.js house generated from the repo's real structure. Room size is lines of code, fog is test coverage, flickering lights are churn, cobwebs are age.
- **Talk to the dead:** ask any question about a file or function. Retrieval runs over commits, PR threads, issues and code (hybrid keyword plus vector search); answers carry clickable citations or the ghost refuses to speak.
- **Haunting detector:** static analysis finds unreachable code, unused exports, abandoned TODOs and stale feature flags, and spawns a ghost for each one.
- **Exorcism:** pick a ghost and the agent opens a real pull request that removes it, with tests proving nothing broke.
- **Séance in your editor:** an MCP server so Kiro (or any agent) can ask the mansion questions mid-coding.
- **Voice mode:** speak to a ghost with the browser's free speech recognition and hear it answer back.

The demo moment

Open the React repo, walk into the reconciler room, ask the ghost why a strange check exists, and it quotes the 2017 PR that added it. Then exorcise a dead export and show the merged PR.

Build sheet

Stack	Next.js, Three.js / React Three Fiber, GitHub REST + GraphQL API (free tier), tree-sitter for parsing, transformers.js embeddings running locally (bge-small), SQLite + sqlite-vec or Supabase free tier with pgvector, Web Speech API, Claude through Kiro for development.
Free resources	GitHub API, any public repo as demo data (React, Django, Linux kernel subtrees), open-source embedding models, free Vercel / Supabase tiers.
Kiro usage	• Specs for each subsystem: ingestion, retrieval, mansion layout, exorcism PR flow. • Hooks: on save, regenerate the mansion layout tests; on new tool, update the MCP schema docs. • Property-based tests: citations must always resolve to a real commit; layout never overlaps rooms. • Custom MCP server built and consumed inside Kiro itself during development.
Why it is heavy	Code parsing across languages, git history mining, hybrid search, 3D procedural layout, PR automation and an MCP server: six real systems stitched into one product.
Main risk	Large repos are slow to ingest. Cap the demo to curated repos and pre-index them.

IDEA 2 | RESURRECTION

DEAD AIR

A spirit box for the real airwaves: tune live shortwave radio from anywhere on Earth and let AI tell you what you are hearing.

The idea

Shortwave radio is the dead technology people forgot is still alive: numbers stations, ships, weather fax, amateur operators bouncing signals across continents, emergency nets during disasters. Hundreds of volunteer-run web receivers around the world stream it for free, but the experience is stuck in 2005 and almost nobody can tell what they are listening to. Dead Air is a haunted radio console that tunes those receivers, decodes what it can, transcribes and translates voices, identifies the signal type and plots every catch on a glowing world map.

What it does

- **Spirit box tuner:** a skeuomorphic, CRT-lit radio with a live waterfall display. Spin the dial, and the app picks the best public receiver near the signal's likely origin.
- **Signal identification:** a small classifier trained on public signal samples labels what is on screen: voice, Morse, RTTY, SSTV, FT8, weather fax, digital noise.
- **Decoders in the browser:** Morse to text, SSTV to images (pictures coming from the sky), FT8 spots on the map, all running in WebAssembly.
- **Transcribe and translate:** Whisper running locally turns voice traffic into text, translated into English, with a live caption feed.
- **Ghost log:** every catch saved with time, frequency, receiver, decoded content and a map pin, building a shared haunted atlas of the airwaves.
- **Disaster mode:** watches known emergency frequencies and raises an alert when traffic spikes, useful for hobbyists and relief volunteers.

The demo moment

Spin the dial live during the video, land on a real Morse signal from Europe, watch it decode, then catch an SSTV image arriving line by line from a ham operator.

Build sheet

Stack	SvelteKit or Next.js, Web Audio API, WebAssembly DSP (Rust compiled to WASM), whisper.cpp / transformers.js Whisper, ONNX Runtime Web for the classifier, MapLibre GL, public KiwiSDR / WebSDR receivers.
Free resources	Public KiwiSDR and WebSDR networks, the open signal identification wiki (sigidwiki) samples, open-source fldigi and SSTV decoder references, free OpenStreetMap tiles, open Whisper models.
Kiro usage	• Specs per decoder with exact acceptance tests (a known Morse clip must decode to known text). • Property-based tests on DSP code: filters stay stable, decoders never crash on random noise. • Steering docs that pin audio constraints (sample rates, buffer sizes) so generated code stays correct. • Hooks that re-run the decoder test bench whenever DSP files change.
Why it is heavy	Real-time signal processing, multiple decoders, an ML classifier, local speech recognition and a live map. Almost no hackathon team attempts DSP.
Main risk	Public receivers have per-user limits and their own terms. Respect them, cache demo recordings and contact a few operators early.

IDEA 3 | RESURRECTION

NECROPOLIS

A graveyard of the dead web where you dig up any lost website and an agent brings it back to life as working, modern code.

The idea

Link rot is erasing the internet: a large share of links cited in news articles, court opinions and papers no longer work, and early web culture survives only as broken snapshots. Necropolis is a moonlit cemetery where each tombstone is a dead website. Dig one up and the resurrection begins: the agent pulls every archived snapshot, reconstructs the pages, repairs the broken links from other captures, revives Flash content with an open emulator and outputs a modern, accessible, deployable version of the site, while keeping the original look intact.

What it does

- **The cemetery:** browse famous dead sites by era, or type any URL to find its grave. Tombstones show birth and death dates taken from archive snapshots.
- **Resurrection pipeline:** the archive's CDX index finds every capture, a crawler rebuilds the site graph, and missing assets are recovered from other snapshots.
- **Flash is not dead:** embedded SWF games and animations play again through the open-source Ruffle emulator.
- **Modernize, do not erase:** output a static site with semantic HTML, alt text written by a vision model, mobile layout and working links, toggled side by side with the original.
- **Autopsy report:** what was lost, what was recovered, which links still point nowhere and where to find their contents now.
- **For researchers:** repair a list of dead citations in a paper or article in one go.

The demo moment

Dig up a 1999 fan site, watch the resurrection animation run through each stage, then play its long-dead Flash game in the browser and flip to the modern version on a phone.

Build sheet

Stack	Next.js, Internet Archive Wayback CDX and availability APIs, Playwright for rendering, Ruffle (WASM Flash emulator), a vision model for alt text, Astro for the generated sites, Cloudflare Pages free tier for one-click redeploys.
Free resources	Internet Archive APIs, Ruffle, GeoCities-era public archives, Common Crawl index, free hosting tiers.
Kiro usage	• Spec-driven pipeline: discovery, reconstruction, repair, modernization, each with acceptance tests. • Hooks that snapshot-test the visual output of a resurrected site after every change. • A custom MCP server wrapping the Wayback API so Kiro can query the archive while building. • Property-based tests: every rewritten link must resolve or be reported, never silently dropped.
Why it is heavy	Large-scale archive querying, crawling, asset recovery, emulation and code generation of whole sites.
Main risk	Copyright on revived sites. Resurrect for preview and personal use, link to the archive and only redeploy sites you own or that are public domain.

IDEA 4 | COSTUME CONTEST

POSSESSED

Your browser is haunted. See every tracker as a creature crawling across the page, and exorcise it.

The idea

Most people have no idea how many companies watch them on a single page. Privacy tools show a number in a badge, and nobody looks at it. Possessed is a browser extension that makes surveillance visible: every tracker, fingerprinting script and data broker call becomes a creature on the page. Its size is how much data it takes, its color is who it reports to, and it follows your cursor when it is recording what you do. Exorcise it with a click to block it, and see the page lighten.

What it does

- **Live haunting overlay:** creatures spawn as network requests happen, sized and colored by tracker company and data type.
- **Who is watching:** click a creature for a plain-English card: the company, what it collects, where the data goes, and which other sites share it.
- **Fingerprint mirror:** a mirror shows what your browser reveals (fonts, canvas, GPU) and how unique that makes you.
- **Exorcism:** block trackers one by one or banish them all, using declarative blocking rules that keep pages working.
- **Haunting history:** a dashboard of the creatures that followed you this week, across sites, as a family tree of data brokers.
- **Accessible by default:** reduced-motion and screen reader modes describe the haunting in words.

The demo moment

Open a big news site and the page fills with creatures within seconds. Click one to see it reports to many companies, then banish them all and watch the page clear.

Build sheet

Stack	Chrome / Firefox extension (Manifest V3), declarativeNetRequest, PixiJS or Canvas for the creatures, IndexedDB for history, a small local model for the plain-English summaries, plus a companion website.
Free resources	DuckDuckGo Tracker Radar dataset, EasyList / EasyPrivacy filter lists, Disconnect tracking protection list, open fingerprinting research.
Kiro usage	• Steering docs for the visual language so every creature and animation stays consistent. • Specs for request capture, attribution, rendering and blocking. • Hooks that run accessibility checks and screenshot diffs on every UI change. • Property-based tests: blocking rules must never break a list of top sites.
Why it is heavy	Real-time network attribution, a full extension, a custom creature animation system, a privacy knowledge base and a companion dashboard.
Main risk	The overlay must stay smooth on heavy pages. Budget time for performance and test on slow laptops.

IDEA 5 | SKELETON CREW

WITCHING HOUR

A lean skeleton for agents that watch the world while you sleep, proven with two very different apps.

The idea

Everyone wants agents that watch something and act when it matters, and everyone rebuilds the same plumbing: connect a source, queue events, ask a model whether it matters, take an action, log why. Witching Hour is that plumbing as a small, readable template: sources, a queue, a triage step with structured output, actions, a spend cap and an audit trail. To show its range, two apps are built on the same skeleton.

What it does

- **The skeleton:** pluggable sources, a durable queue with retries and dead letters, one structured model call per event, actions, cost caps and a full audit log, in a few hundred clear lines.
- **App 1, Night Watch:** watches public earthquake, space weather and wildfire feeds, decides what is worth waking you for, and narrates it in a spooky late-night radio host voice on a live map.
- **App 2, Poltergeist:** haunts your RAG chatbot overnight, firing adversarial questions to find data leaks, including leaks that only appear when harmless facts are combined, and leaves a morning report with fixes.
- **Graveyard shift dashboard:** one shared UI shows every agent, every event, why it acted, and what it cost.
- **Bring your own app:** a guided generator scaffolds a third agent from a short description.

The demo moment

Fast-forward one night: Night Watch catches a real earthquake and wakes you with the narration, while Poltergeist finds a leak in the demo chatbot and opens the fix.

Build sheet

Stack	TypeScript, a lightweight queue (SQLite-backed or pgmq on Supabase free tier), structured tool calling, MapLibre for Night Watch, a demo RAG app for Poltergeist to attack.
Free resources	USGS earthquake feed, NOAA space weather, NASA FIRMS wildfire data, open-source RAG stack for the target app.
Kiro usage	• Kiro Crew or custom agents for the overnight runs, which also opens the Kiro Crew bonus prize. • Specs for the skeleton contract, so both apps are generated against the same interface. • Hooks that run both apps' test suites whenever the skeleton changes, proving it stays reusable. • Property-based tests on the queue: no event lost, none processed twice.
Why it is heavy	A reusable framework plus two complete, unrelated products, an attack engine and a shared dashboard.
Main risk	Skeleton Crew judges want the template to be lean. Keep the core small and put the size in the two apps.

SIDE BY SIDE

Comparing the five

Rough calls to help the choice. Effort assumes two people for five weeks.

	Category	Wow factor	Real-world use	Hardest part	Effort
Séance	Frankenstein + Costume	3D haunted mansion of real code	Onboarding, code archaeology	Ingesting big repos fast	High
Dead Air	Resurrection	Live radio decoded from the sky	Radio hobby, disaster monitoring	Real-time DSP and decoders	Very high
Necropolis	Resurrection	Dead Flash sites playing again	Link rot, digital preservation	Reconstructing messy archives	High
Possessed	Costume Contest	Trackers crawling over pages	Everyday privacy	Smooth animation on heavy pages	Medium-high
Witching Hour	Skeleton Crew	Agents working overnight	Reusable agent plumbing	Keeping the core lean	High

Five-week shape for whichever one we pick

Week	Goal
1 (Oct 30 - Nov 6)	Request credits, set up repo and /.kiro steering docs, write specs, build the core pipeline end to end, even if ugly.
2 (Nov 7 - 13)	Make every core feature work on real data. Hooks and property-based tests running.
3 (Nov 14 - 20)	The signature UI and the demo moment. MCP or custom agents in place.
4 (Nov 21 - 27)	Polish, performance, accessibility, deploy a public URL, freeze features.
5 (Nov 28 - Dec 3)	Video, description, Kiro write-up, social post and blog. Submit by Dec 3, a day early.